

Лабораторная работа №2. Знакомство с Flutter

Цель работы: Познакомиться с основным инструментом кроссплатформенной разработки — Flutter. Создать и запустить первый Flutter-проект в браузере Chrome, изучить структуру проекта и базовые концепции фреймворка — виджеты и дерево виджетов.

Необходимые инструменты:

- **Flutter 3.35+**
- **VS Code** (рекомендуется для лабораторной работы), либо **IntelliJ IDEA / Android Studio** с установленным плагином **Flutter**
- **Git**
- **Браузер Google Chrome / Edge**

*P.S. Если Flutter ещё не установлен — видео-инструкция по установке доступна на сайте курса. После установки обязательно выполните **flutter doctor** для проверки окружения.*

Теоретическая часть. Что такое Flutter?

Flutter — это открытый кроссплатформенный UI-фреймворк, разработанный компанией **Google**. Он позволяет писать один код на языке **Dart** и собирать из него нативные приложения для шести платформ одновременно: Android, iOS, Web, Windows, macOS и Linux.

В отличие от React Native или Ionic, Flutter не использует нативные компоненты платформы и не создаёт мост к ним. Вместо этого Flutter рисует весь интерфейс самостоятельно через собственный графический движок **Skia / Impeller**. Это даёт пиксельно-точный одинаковый UI на любой платформе и высокую производительность (60/120 fps).

Сравнение с Jetpack Compose

Вы уже работали с Jetpack Compose — декларативным UI-фреймворком для Android. Flutter построен на той же концепции: UI описывается как функция от состояния, а не через мутацию View-объектов. Это делает переход значительно проще, чем кажется на первый взгляд.

Виджет	Аналог в Jetpack Compose (Kotlin)	Описание
Text	Text	Отображение текста
ElevatedButton	Button / ElevatedButton	Кнопка с эффектом тени
Column / Row	Column / Row	Вертикальное / горизонтальное расположение дочерних элементов
Box / Container	Box	Универсальный контейнер с отступами, цветом, скруглением
Image	Image / AsyncImage (Coil)	Отображение изображения
Scaffold	Scaffold	Базовый «каркас» экрана: TopBar, content и т.д.
StatefulWidget + setState()	remember { mutableStateOf() } + State	Управление изменяемым состоянием с автоматической перекomпозицией UI
StatelessWidget	Composable-функция без state	UI без состояния, зависит только от входных параметров
Navigator / Router	NavController (Navigation Compose)	Навигация между экранами

Ключевое сходство между **Flutter** и **Jetpack Compose**: оба фреймворка декларативны — вы описываете, как должен выглядеть UI при данном состоянии, а фреймворк сам решает, что перерисовать. В Compose это называется перекomпозицией, во Flutter — перестройкой дерева виджетов. Главное практическое отличие: в Compose состояние хранится через `remember { mutableStateOf() }`, а во **Flutter** — через поле в State-классе с вызовом `setState()`. В остальном логика работы с состоянием идентична.

Где используется Flutter

Flutter применяется в коммерческой разработке по всему миру. Несколько известных приложений, созданных на Flutter:

- **Google Pay** — платёжное приложение Google;
- **BMW App** — официальное мобильное приложение BMW;
- **Alibaba Xianyu** — крупнейшая платформа перепродажи в Китае;
- **eBay Motors** — приложение для продажи автомобилей;
- **Rive** — профессиональный инструмент для создания анимаций.

Что такое виджет?

Виджет (Widget) — это базовый строительный блок UI во Flutter. Абсолютно всё, что вы видите на экране (и даже то, что не видите) — это виджет: текст, кнопка, отступ, выравнивание, анимация, даже само приложение.

Виджеты бывают двух видов:

- **StatelessWidget** — виджет без состояния. Отрисовывается один раз на основе входных данных и не меняется. Аналог — обычный View в Android, который не изменяется после создания.

- **StatefulWidget** — виджет с состоянием. Может перестраиваться при вызове setState(). Аналог — ViewModel + LiveData в Android: при изменении данных UI обновляется автоматически.

Виджеты образуют дерево (Widget Tree) — иерархическую структуру, где каждый виджет может содержать дочерние виджеты. Это похоже на иерархию View в Android, но во Flutter дерево перестраивается при каждом изменении состояния, что делает разработку предсказуемой.

Часть 1: Подготовка проекта

Шаг 1. Проверка инструментов

1. Откройте терминал: **Git Bash / PowerShell** или **Linux terminal**
2. Убедитесь, что Flutter установлен — выполните команду:

flutter --version

```
└─$ flutter --version
Flutter 3.41.2 • channel stable • https://github.com/flutter/flutter.git
Framework • revision 90673a4eef (2 дня назад) • 2026-02-18 13:54:59 -0800
Engine • hash d96704abcce17ff165bbef9d77123407ef961017 (revision 6c0baaebf7) (2
days ago) • 2026-02-18 19:22:23.000Z
Tools • Dart 3.11.0 • DevTools 2.54.1
```

Ожидаемый результат — отображение установленной версии SDK. Если команда не найдена — Flutter не добавлен в PATH. Следуйте видео-инструкции по установке на сайте курса или выполните flutter doctor для диагностики.

3. Проверьте доступные устройства:

flutter devices

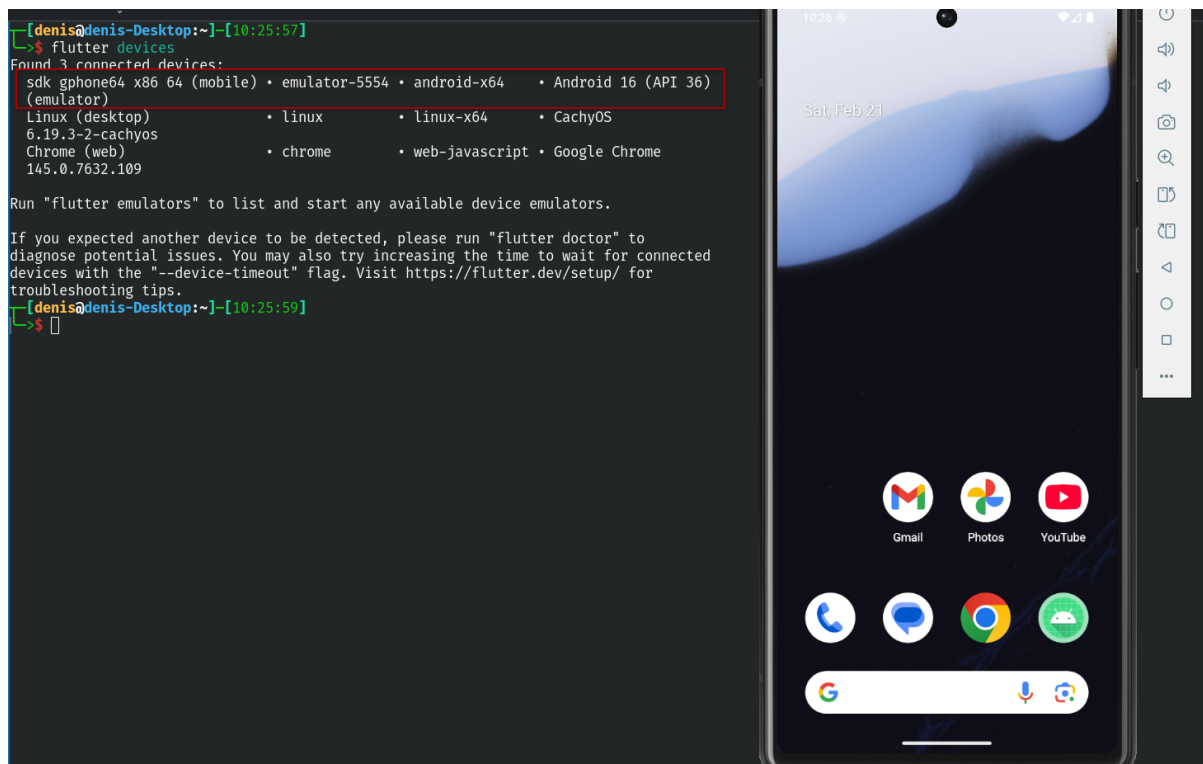
Ожидаемый вывод:

```
└─$ flutter devices
Found 2 connected devices:
  Linux (desktop) • linux • linux-x64 • CachyOS 6.19.3-2-cachyos
  Chrome (web) • chrome • web-javascript • Google Chrome 145.0.7632.109

Run "flutter emulators" to list and start any available device emulators.
```

P.S. На Windows по умолчанию будут доступны Chrome и Microsoft Edge (если установлены).

При желании можно также запустить Android-эмулятор из Android Studio — тогда он появится в списке устройств:



В данной лабораторной работе мы работаем в браузере **Chrome**. Это самый быстрый способ запуска Flutter без настройки эмуляторов — горячая перезагрузка (**Hot Reload**) работает мгновенно.

Шаг 2. Создание проекта

1. Перейдите в папку с вашими проектами по **Flutter**:

cd Documents/Flutter (как пример)

2. Создайте новый проект:

flutter create first_flutter_app

Создание займёт 30–60 секунд (если только у вас не мобильный интернет). Flutter сформирует полную структуру проекта, установит зависимости и создаст заготовку приложения-счётчика.

3. Перейдите в папку проекта:

cd first_flutter_app

4. Создайте папку для скриншотов:

mkdir img

5. Посмотрите структуру проекта:

ls -la

```

$ ls -la
drwxr-xr-x  - denis 21 фев 10:31 .
drwxr-xr-x  - denis 21 фев 10:30 ..
drwxr-xr-x  - denis 21 фев 10:30 .dart_tool
drwxr-xr-x  - denis 21 фев 10:30 .idea
drwxr-xr-x  - denis 21 фев 10:30 android
drwxr-xr-x  - denis 21 фев 10:31 img
drwxr-xr-x  - denis 21 фев 10:30 ios
drwxr-xr-x  - denis 21 фев 10:30 lib
drwxr-xr-x  - denis 21 фев 10:30 linux
drwxr-xr-x  - denis 21 фев 10:30 macos
drwxr-xr-x  - denis 21 фев 10:30 test
drwxr-xr-x  - denis 21 фев 10:30 web
drwxr-xr-x  - denis 21 фев 10:30 windows
-rw-r--r-- 703 denis 21 фев 10:30 .gitignore
-rw-r--r-- 1,7k denis 21 фев 10:30 .metadata
-rw-r--r-- 1,4k denis 21 фев 10:30 ! analysis_options.yaml
-rw-r--r-- 842 denis 21 фев 10:30 first_flutter_app.iml
-rw-r--r-- 6,1k denis 21 фев 10:30 pubspec.lock
-rw-r--r-- 3,8k denis 21 фев 10:30 ! pubspec.yaml
-rw-r--r-- 639 denis 21 фев 10:30 README.md

```

Структура проекта

Файл / Папка	Назначение
.dart_tool/	Служебная папка Dart/Flutter, создаётся автоматически. Хранит кэш сборки и конфигурацию инструментов. Не редактировать вручную.
.idea/	Настройки IDE (IntelliJ IDEA / Android Studio). Содержит конфигурации запуска и стиль кода.
android/	Нативный Android-проект. Flutter встраивает своё приложение в него при сборке под Android.
ios/	Нативный iOS-проект. Аналогично android/, нужен при сборке под iPhone/iPad.
linux/ macos/ windows/	Нативные проекты для настольных платформ. Используются при сборке под соответствующую ОС.
web/	Точка входа для веб-сборки. Содержит index.html, манифест и иконки.
lib/	Главная папка с вашим Dart-кодом. Здесь будет весь ваш код приложения. Файл lib/main.dart — стартовая точка запуска.
test/	Папка для unit- и widget-тестов.
img/	Папка для скриншотов (создана вами в ходе работы).
pubspec.yaml	Манифест проекта: название, версия, зависимости (пакеты), подключение assets (изображений, шрифтов). Аналог build.gradle в Android.
pubspec.lock	Автоматически генерируется на основе pubspec.yaml. Фиксирует точные версии всех зависимостей.
.gitignore	Список файлов и папок, которые Git не отслеживает (кэш, временные файлы сборки).
analysis_options.yaml	Настройки статического анализатора Dart (линтер). Определяет правила проверки кода.
README.md	Описание проекта. Отображается на GitHub/GitLab как главная страница репозитория.

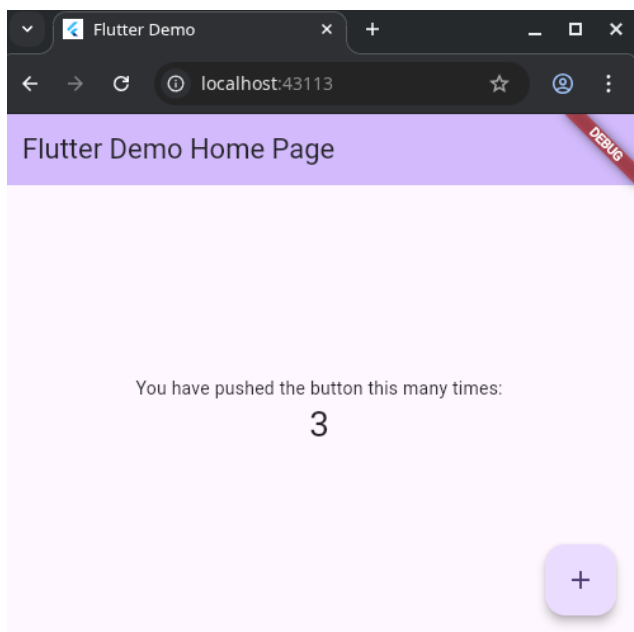
💡 Основная папка, с которой вы будете работать — **lib/**. Остальные папки платформ (android/, ios/, windows/ и т.д.) генерируются автоматически и на начальном этапе не требуют изменений.

Шаг 3. Запуск проекта

1. Запустите приложение в Chrome:

flutter run -d chrome

Первый запуск может занять 1–2 минуты. Flutter собирает проект для браузера. Последующие запуски будут значительно быстрее благодаря кэшированию.



2. В браузере откроется демо-приложение — простой счётчик кликов. Это стандартная заготовка Flutter. Нажмите кнопку «+» несколько раз и убедитесь, что счётчик увеличивается.

3. В терминале появятся горячие клавиши для управления запущенным приложением:

```
Flutter run key commands.
r Hot reload. 🔥🔥🔥
R Hot restart.
h List all available interactive commands.
d Detach (terminate "flutter run" but leave application running).
c Clear the screen
q Quit (terminate the application on the device).
```

📸 **СКРИНШОТ 1:** Сделайте скриншот браузера с работающим счётчиком И окна терминала с логом запуска. Сохраните в папку **img** под именем **step1_ВашаФамилия.png**

4. После скриншота нажмите **q** в терминале для выхода из режима запуска.

Шаг 4: Инициализация Git

1. Откройте приложение в VS Code:

code .

2. Откройте в нём встроенный терминал

3. Инициализируйте репозиторий с основной веткой main:

git init -b main

4. Настройте **.gitignore** — исключим ненужные файлы. Поскольку в этой работе мы разрабатываем только под Web, папки для других платформ и системные папки нам в репозитории не нужны. Откройте **файл .gitignore** и добавьте в конец:

```
# Платформы, которые не используем в данной работе
android/
ios/
linux/
macos/
windows/
# IDE и системные файлы
test/
.idea/
*.iml
.DS_Store
Thumbs.db
```

Зачем это делать? Папки android/, ios/ и другие содержат тысячи автогенерируемых файлов. Добавление их в Git засоряет историю коммитов и замедляет работу. В реальном проекте их добавляют только если проект планируется к публикации на соответствующей платформе.

5. В терминале добавьте все файлы в индекс (с учётом обновлённого .gitignore):

git add .

6. Сделайте первый коммит:

git commit -m "Initial commit: create Flutter project"

Шаг 5. Подключение к GitHub

1. Создайте новый **public**-репозиторий на GitHub:

Flutter_Lab2

Без **README**, **.gitignore** и **License**.

2. Привяжите локальный репозиторий:

git remote add origin <URL_вашего_репозитория>

git push -u origin main



СКРИНШОТ 2: Сделайте скриншот терминала с выводом пуша. Сохраните в **img/step2_ВашаФамилия.png**

Часть 2. Знакомство с проектом и инструментами VS Code

Настройка `dart.lineLength` в `settings.json`

На первой лабораторной вы уже правили `settings.json` для Dart. Сейчас нам нужно изменить длину строки — при более коротком значении VS Code будет автоматически переносить аргументы виджетов на новые строки, и дерево виджетов станет читаться значительно лучше.

Откройте `settings.json` (Ctrl+Shift+P → Open User Settings JSON) и замените или добавьте:

```
"dart.lineLength": 50,
```

Лучше выбирать в районе 45-50. Сохраните файл.

Шаг 1. Изучаем `main.dart` — убираем лишнее

Откройте файл `lib/main.dart`. На первый взгляд он выглядит громоздко. Но большую часть занимают комментарии, которые Flutter добавляет автоматически как подсказки для новичков.

Задание: удалите все комментарии из файла вручную. Комментарии в Dart — это строки, начинающиеся с `//` или блоки `/* ... */`. Удалите их все, включая пустые строки, которые после этого остались.

После очистки в файле останется **намного меньше строк** — и вы увидите, что код на самом деле совсем не страшный. Это три класса и пара методов.

Сделайте коммит:

```
git add lib/main.dart
```

```
git commit -m "Remove comments from main.dart"
```

Шаг 2. Способы запуска приложения в VS Code

VS Code с плагином **Flutter** даёт несколько способов запустить приложение. Разберём все четыре — они пригодятся в разных ситуациях.

Перед запуском убедитесь, что в правом нижнем углу VS Code выбрано устройство **Chrome**. Кликните на строку с устройством и выберите **Chrome** из списка:



Способ 1 — CodeLens: кнопка **Run** над `main()`

Откройте `lib/main.dart`.

Прямо над строкой `void main()` появляются две ссылки Run | Debug:



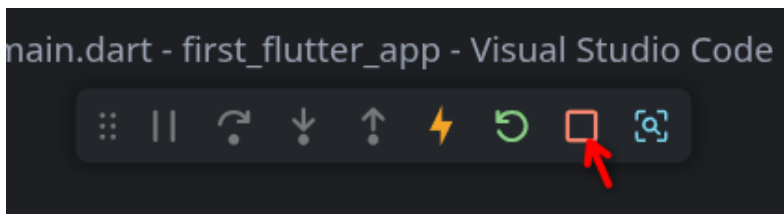
Это **CodeLens** — встроенная функция VS Code, которая показывает действия прямо в коде.

- **Run** — запускает без отладчика (быстрее, нет точек останова)
- **Debug** — запускает с отладчиком

Нажмите **Run**. Приложение откроется в Chrome.

Способ 2 — **Ctrl+F5 (Run Without Debugging)**

Закройте приложение (нажмите кнопку **Stop** в VS Code).



Нажмите **Ctrl+F5**. Это стандартное сочетание VS Code для запуска **без отладчика** — работает так же, как Run в CodeLens, но быстрее доступно с клавиатуры.

Используйте этот способ когда просто хотите посмотреть результат — без лишних накладных расходов отладчика.

Способ 3 — **F5 (Debug Mode)**

Нажмите **F5**. Приложение запустится в **режиме отладки** — в VS Code активируется панель Debug, появятся кнопки паузы, шага по коду и т.д.

В режиме отладки вы можете:

- ставить **точки останова** (breakpoints) — кликните на номер строки слева
- смотреть значения переменных в панели **Variables**
- выполнять код пошагово

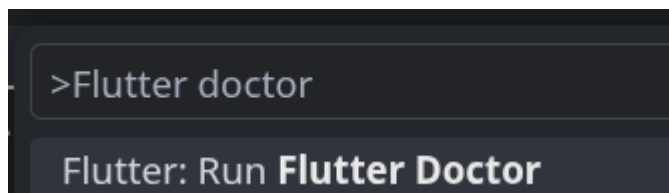
Для Flutter-разработки режим отладки особенно полезен при работе со сложной логикой и состоянием.

Шаг 3. Command Palette, горячие клавиши, DevTools и Flutter Inspector

Ctrl+Shift+P → команда Flutter

Нажмите **Ctrl+Shift+P** — откроется **Command Palette** (палитра команд).

Введите **Flutter** — увидите список доступных команд. Введите **Flutter doctor**:



У вас появится вывод того что вы делали ранее в терминале командой.

Command Palette — самый гибкий способ: отсюда можно менять устройство, открывать DevTools, запускать эмулятор и т.д.

3.1. Запуск через терминал и горячие клавиши

Остановите приложение, если оно запущено. Теперь запустим его через терминал — именно в этом режиме работают горячие клавиши управления:

flutter run -d chrome

После запуска в терминале увидите:

```
Flutter run key commands.
r Hot reload. 🔥🔥🔥
R Hot restart.
h List all available interactive commands.
d Detach (terminate "flutter run" but leave application running).
c Clear the screen
q Quit (terminate the application on the device).
```

3.2. Hot Reload vs Hot Restart — в чём разница?

Это один из самых важных инструментов Flutter-разработчика. Понимание разницы сэкономит много времени.

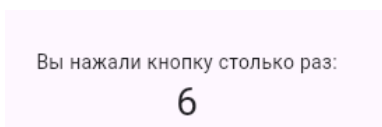
	Hot Reload (r)	Hot Restart (R)
Что делает	Подставляет новый код в работающее приложение	Перезапускает приложение с нуля
Состояние	Сохраняется (счётчик не сбрасывается)	Сбрасывается (как свежий запуск)
Скорость	~300 мс	~2–5 сек
Когда нужен	Изменения UI, текст, стили, вёрстка	Изменения в <code>initState()</code> , конструкторах, глобальных переменных

Попробуем на практике:

Тест Hot Reload:

1. Нажмите кнопку «+» в приложении несколько раз — счётчик, например, показывает 6.
2. Откройте `lib/main.dart`.
3. Найдите строку:
'You have pushed the button this many times:'
4. Замените на:
'Вы нажали кнопку столько раз:'
5. Сохраните (**Ctrl+S**).
6. В терминале нажмите **r**.

Результат:



Текст в браузере обновился мгновенно, а счётчик по-прежнему показывает 6 — состояние сохранилось. Это и есть **Hot Reload**.

Тест Hot Restart:

Теперь нажмите в терминале **R** (заглавная).

Результат: приложение перезапустилось, счётчик сбросился в 0. Текст изменённый остался — код новый, но состояние потеряно.

Обратите внимание на время выполнения:



Вывод: используйте **r** для правки UI во время разработки — это экономит время. **R** нужен когда поменяли что-то в логике инициализации и Hot Reload не подхватил изменения.

3.3. Flutter DevTools в браузере

DevTools — встроенный набор инструментов для инспекции Flutter-приложения прямо в браузере.

При запуске через `flutter run` в терминале появляется ссылка:

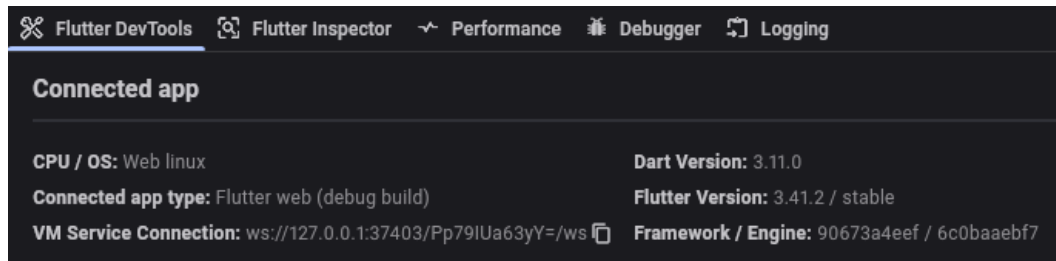
The Flutter DevTools debugger and profiler on Chrome is available at:

<http://127.0.0.1:9100/...>

The Flutter DevTools debugger and profiler on Chrome is available at:
<http://127.0.0.1:37403/Pp79IUa63yY=/devtools/?uri=ws://127.0.0.1:37403/Pp79IUa63yY=/ws>

Либо откройте через Command Palette: **Ctrl+Shift+P** → **Flutter: Open DevTools**.

В DevTools доступны вкладки:



- **Flutter Inspector** — визуальное дерево виджетов. Можно кликнуть на элемент на экране и сразу увидеть какой виджет за него отвечает и какие у него свойства.
- **Performance** — график FPS и нагрузки на рендеринг.
- **Debugger** — в нашем случае мы видим вывод - *Инструменты разработчика скрывают вкладку «Отладчик», если приложение было запущено из VS Code, поскольку VS Code имеет встроенный отладчик.*
- **Logging** — логи и вывод `print()`.

Задание: откройте DevTools, перейдите на вкладку **Flutter Inspector** и найдите в дереве виджет **Text** со счётчиком. Посмотрите его свойства справа.

 **СКРИНШОТ 3:** Скриншот **DevTools** с открытым **Flutter Inspector**. Сохраните в [img/step3_ВашаФамилия.png](#)

3.4. Как работает `setState()` — разбираем на счётчике

Прежде чем мы удалим весь код и начнём писать своё приложение — разберём как работает единственная по-настоящему важная вещь в этом демо: **управление состоянием**.

Найдите в `main.dart` этот метод:

```
void _incrementCounter() {  
  setState(() {  
    _counter++;  
  });  
}
```

Вот что происходит при нажатии кнопки «+»:

1. Вызывается `_incrementCounter()`

2. Внутри вызывается `setState()`
3. Flutter получает сигнал: «состояние изменилось, нужна перерисовка»
4. Метод `build()` вызывается заново
5. Виджет `Text('$counter')` получает новое значение и обновляется на экране

Ключевое правило: никогда не меняйте переменные состояния напрямую без `setState()` — Flutter не узнает об изменении и UI не обновится. Это самая частая ошибка новичков.

По аналогии с Jetpack Compose — `setState()` это `mutableStateOf()`: в обоих случаях фреймворк сам решает что перерисовать.

Сделайте **КОММИТ** перед тем как мы удалим демо-код:

git add .

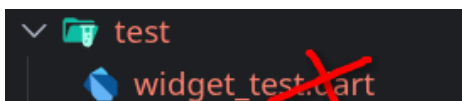
git commit -m "Explore project structure and VS Code tools"

Часть 3. Пишем приложение с нуля: первые виджеты

Шаг 1. Точка входа и первый виджет

Чистый старт

1. Откройте `lib/main.dart` и удалите **весь** код в файле — он должен стать полностью пустым.
2. Также удалите файл `test/widget_test.dart` — он написан под старый код счётчика и сейчас будет только мешать.



Именно написав всё с нуля, вы поймёте зачем нужна каждая строчка. Готовый шаблон Flutter скрывает эту логику за комментариями и заготовками.

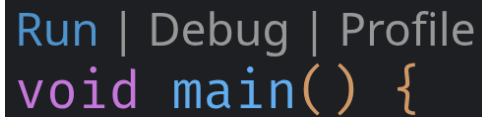
Шаг 2. `main()` — точка входа

Любая программа на **Dart** начинается с функции `main()` — это точка входа, с которой стартует выполнение кода. Вы уже встречали это в Kotlin (`fun main()`) и C# (`static void Main()`). Во Flutter всё то же самое.

1. Напишите в пустом `main.dart`:

```
void main() {  
  
}
```

Обратите внимание: как только вы написали `main()`, прямо над ней в VS Code появились три кнопки:



Это **CodeLens**, который вы уже знаете. Но теперь появился третий режим — **Profile**:

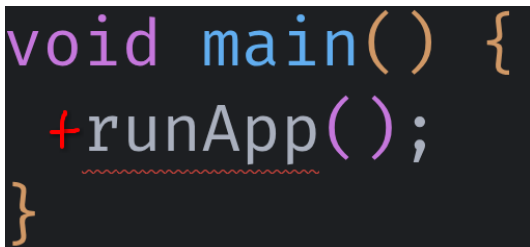
Режим	Клавиша	Назначение
Run	Ctrl+F5	Запуск без отладчика. Используйте для обычной разработки — быстрее всего.
Debug	F5	Запуск с отладчиком. Позволяет ставить точки останова и смотреть переменные.
Profile	—	Запуск в режиме профилирования. Используется для измерения производительности: FPS, потребление памяти, время рендеринга. В обычной учебной разработке не нужен.

Profile-режим недоступен для веб-платформы в полном объёме — он ориентирован на мобильные приложения. Для веба используйте DevTools.

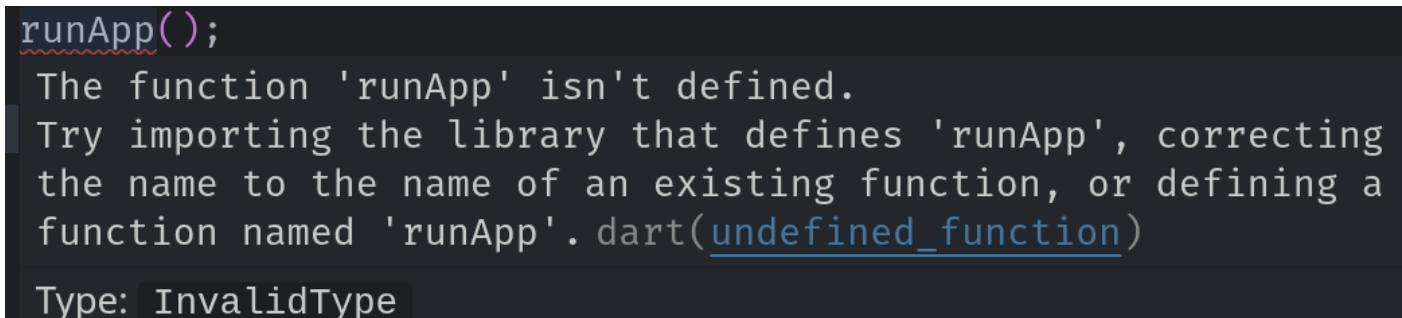
Шаг 3. `runApp()` — запуск интерфейса

Сама по себе функция `main()` просто запускает программу. Чтобы Flutter показал что-то на экране, нужна специальная функция — `runApp()`. Её единственная задача — взять виджет и отобразить его на весь экран.

1. Добавьте её внутрь `main()`:



2. Сразу появится ошибка — VS Code подчеркнёт `runApp` красным. Наведите курсор и увидите:



VS Code не знает что такое `runApp` — эта функция определена во Flutter-библиотеке, которую мы ещё не подключили. В Dart, как и в Kotlin и C#, внешние библиотеки нужно явно импортировать.

3. Добавьте в самое начало файла:

```
import 'package:flutter/material.dart';
```

Пакет `flutter/material.dart` — это основная библиотека Flutter, которая содержит:

- саму функцию `runApp()`
- все базовые виджеты: `Text`, `Column`, `Row`, `ElevatedButton` и сотни других
- Material Design — систему визуального оформления от Google, на которой построено большинство Android-приложений

После добавления импорта обратите внимание: `runApp` поменял цвет — VS Code нашёл функцию и теперь подсвечивает её как известный идентификатор.

Шаг 4. Передаём виджет в `runApp()`

Теперь новая ошибка:

```
1 positional argument expected by 'runApp', but 0 found.  
Try adding the missing  
argument. dart(not_enough_positional_arguments)
```

`runApp()` ожидает один обязательный аргумент — **виджет**, который нужно отобразить. Flutter полностью построен на виджетах: каждый элемент интерфейса, от кнопки до отступа — это виджет. Весь список с примерами и документацией доступен на официальном сайте:

<https://docs.flutter.dev/ui/widgets>

Для запуска полноценного Flutter-приложения нам нужен виджет `MaterialApp` — он является корневым виджетом и настраивает всё приложение: тему, навигацию, локализацию и многое другое. Без него большинство других виджетов работать не будут.

Передайте его в `runApp()`:

```
runApp(MaterialApp());
```

`MaterialApp()` — это вызов конструктора класса `MaterialApp`. Конструкторы вы уже знаете по Kotlin и C#: они создают объект класса и принимают начальные параметры.

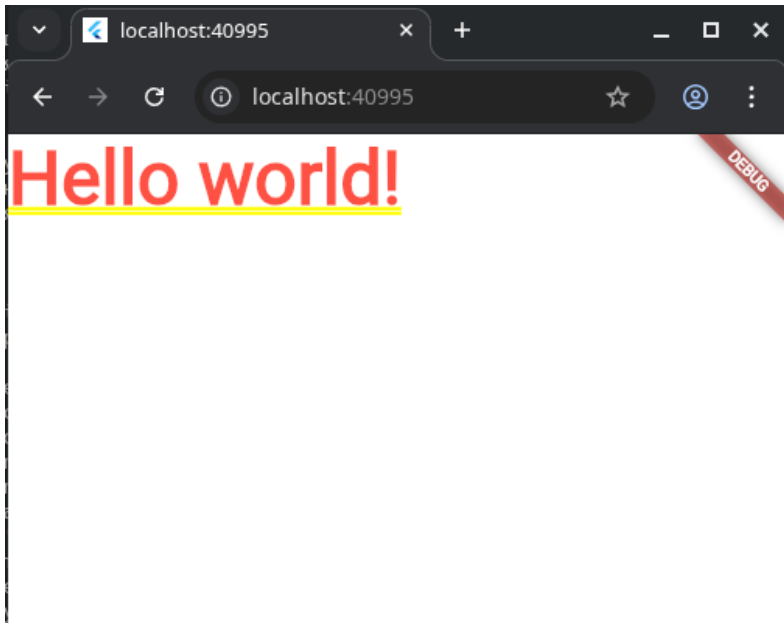
Шаг 5. Первый текст на экране

`MaterialApp` принимает множество именованных параметров. Один из самых важных — `home`: это виджет, который будет показан на главном экране приложения.

Передадим в `home` виджет `Text` с простым текстом:

```
MaterialApp(home: Text("Hello world!"));
```

Запустите приложение любым знакомым способом — например, нажмите **Run** над `main()` или `Ctrl+F5`.



 **СКРИНШОТ 4:** Скриншот браузера с запущенным приложением и текстом "Hello world!". Сохраните в `img/step4_ВашаФамилия.png`

Результат выглядит скромно: текст прижат к левому верхнему углу, нет фона, нет структуры. Но это уже **настоящее работающее Flutter-приложение**, написанное вами с нуля. В следующих шагах мы добавим структуру и сделаем его красивее.

Итог шага

Всего 5 строк — и приложение работает. Запомните эту цепочку:

`main()` → `runApp()` → `MaterialApp()` → `home`: виджет

Это скелет любого Flutter-приложения. Всё остальное — это добавление виджетов внутрь этой структуры.

Сделайте коммит:

`git add .`

`git commit -m "MaterialApp with Hello World"`

Шаг 6. Виджет Scaffold — каркас экрана

Сейчас приложение выглядит сыро: текст прижат к углу, нет никакой структуры. Это потому что у нашего экрана нет «каркаса».

Во Flutter роль каркаса играет виджет **Scaffold**. Он реализует базовую визуальную структуру экрана в стиле **Material Design** и предоставляет готовые слоты для стандартных элементов:

Параметр	Назначение
<code>body</code>	Основное содержимое экрана
<code>appBar</code>	Верхняя панель с заголовком
<code>floatingActionButton</code>	Круглая кнопка действия (FAB)
<code>bottomNavigationBar</code>	Нижняя панель навигации
<code>drawer</code>	Боковое меню

Сейчас нам нужен только **body**. Оберните Text в Scaffold:

```
runApp(MaterialApp(home: Scaffold(body: Text("Hello world!"))));
```

Сохраните файл (Ctrl+S). Автоформатирование применится автоматически — длинная строка превратится в читаемую вертикальную структуру:

```
void main() {
  runApp(
    MaterialApp(
      home: Scaffold(
        body: Text("Hello world!"),
      ), // Scaffold
    ), // MaterialApp
  );
}
```

💡 Обратите внимание как структура кода теперь визуально повторяет дерево виджетов: каждый дочерний виджет сдвинут вправо относительно родителя. Это делает код значительно легче для чтения и отладки.

Перезапустите приложение — визуально пока ничего не изменится, но структура кода стала правильной.

📸 **СКРИНШОТ 5:** Скриншот VS Code с отформатированным кодом. Сохраните в **img/step5_ВашаФамилия.png**

Шаг 7. Выравниваем текст по центру

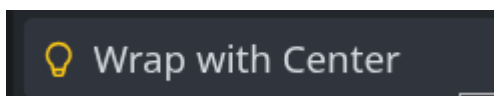
Текст сейчас прижат к левому верхнему углу. Добавим виджет **Center** — он располагает дочерний виджет точно по центру доступного пространства.

Обернуть виджет вручную можно всегда, но VS Code предлагает удобный способ через рефакторинг:

- Кликните на слово **Text** в коде правой кнопкой мыши.
- Выберите **Refactor...** (**Ctrl+Shift+R**).



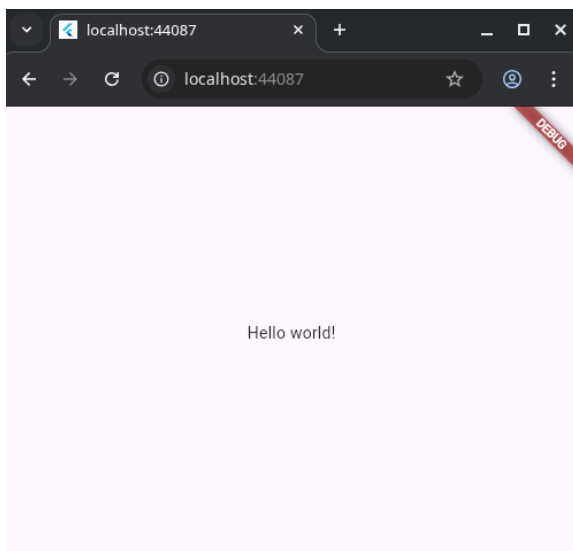
- В появившемся списке выберите **Wrap with Center**.



VS Code автоматически обернёт виджет и применит форматирование.

💡 Запомните этот приём. **Refactor** → **Wrap with Widget** — один из самых часто используемых инструментов при построении UI во Flutter. Через него можно быстро обернуть любой виджет в **Column**, **Row**, **Padding**, **Center** и другие.

Перезапустите приложение и убедитесь что текст теперь выровнен по центру экрана:



📸 **СКРИНШОТ 6:** Скриншот браузера с текстом по центру. Сохраните в **img/step6_ВашаФамилия.png**

Шаг 8. Убираем баннер DEBUG

В правом верхнем углу вы наверняка заметили красную надпись **DEBUG**. Это отладочный баннер, который Flutter показывает в debug-режиме. В готовом

приложении его нет — он исчезает при сборке в release-режиме. Но во время разработки он мешает, поэтому отключим его вручную.

Добавьте параметр `debugShowCheckedModeBanner: false` в `MaterialApp`:

```
MaterialApp(  
  + debugShowCheckedModeBanner: false,  
  home: (Scaffold(  
    // ...  
  ))  
)
```

Посмотрите на дерево виджетов которое у нас получилось:

```
MaterialApp  
├── Scaffold  
│   ├── Center  
│   │   └── Text
```

Это и есть фундаментальный принцип Flutter: UI — это дерево виджетов, где каждый элемент вложен в другой. `MaterialApp` настраивает приложение, `Scaffold` даёт структуру экрана, `Center` управляет позиционированием, `Text` отображает контент.

Сделайте коммит:

```
git add .
```

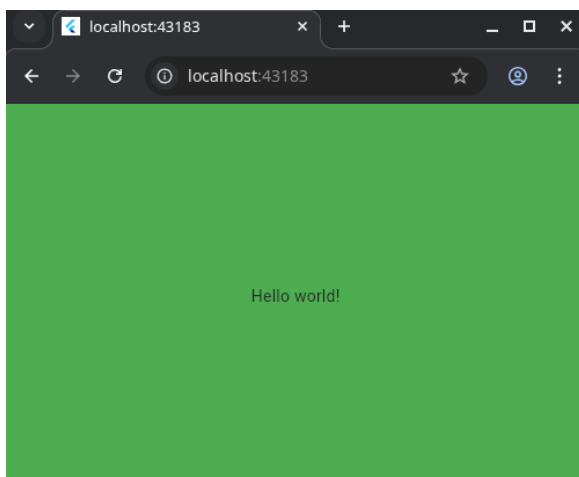
```
git commit -m "Add Scaffold, Center and disable debug banner"
```

Шаг 9. Цвет фона

У `Scaffold` есть параметр `backgroundColor` — он задаёт цвет фона всего экрана. Добавьте его:

```
home: (Scaffold(  
  + backgroundColor: Colors.green,  
  body: Center(  
    // ...  
  ))  
)
```

Сохраните и выполните **Hot Reload** — фон изменился.



Colors — готовая палитра

Colors — это класс Flutter со встроенной коллекцией именованных цветов Material Design: **Colors.red**, **Colors.blue**, **Colors.orange**, **Colors.purple** и десятки других. Удобно для быстрой разработки, но выбор ограничен готовыми вариантами.

Свой цвет через ARGB

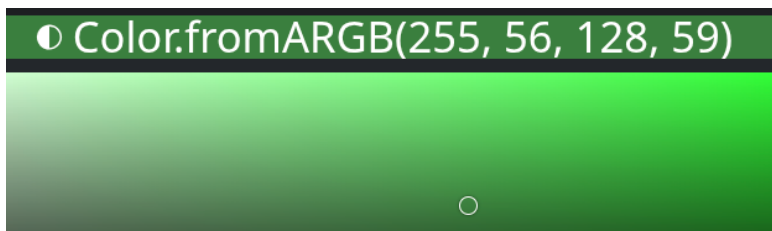
Если нужен точный цвет — используйте модель **ARGB** (Alpha, Red, Green, Blue):

- **Alpha** — прозрачность: 0 = полностью прозрачный, 255 = полностью непрозрачный
- **Red, Green, Blue** — интенсивность каждого канала от 0 до 255

VS Code умеет помочь с выбором цвета: наведите курсор на любое значение цвета в коде — появится цветной квадратик:



Кликните на него, откроется визуальный цветовой пикер. Выберите нужный цвет — код обновится автоматически:



💡 Заметьте ключевое слово **const** — оно говорит Dart что этот объект создаётся на этапе компиляции и никогда не изменится. VS Code часто подсказывает добавить **const** сам — это оптимизация, которая делает приложение чуть быстрее.

📸 **СКРИНШОТ 7:** браузера с изменённым цветом фона. Сохраните в **img/step7_ВашаФамилия.png**

Шаг 10. Градиент фона

Сплошной цвет — скучно. Сделаем градиентный фон.

Убираем backgroundColor

Сначала удалите строку **backgroundColor** которую добавляли в предыдущем шаге — она нам больше не нужна.

```
home: (Scaffold(  
  backgroundColor:  const Color.fromARGB(255, 56, 128, 59),  
  body: Center(  
      
  )  
),
```

Цвет фона теперь будет задаваться через другой механизм.

Оборачиваем Center в Container

Container — один из самых универсальных виджетов Flutter. Он умеет задавать размеры, отступы, границы, тени и — что нужно нам сейчас — **оформление фона** через параметр `decoration`.

Оберните `Center` в `Container` через рефакторинг:

1. Кликните на слово `Center` в коде правой кнопкой мыши.
2. Выберите **Refactor...** (Ctrl+Shift+R).



3. В списке выберите **Wrap with Container**.



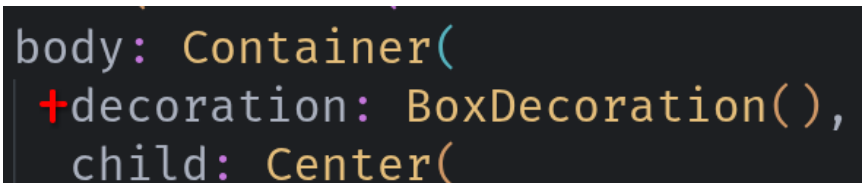
Код станет таким:

```
runApp(  
  MaterialApp(  
    debugShowCheckedModeBanner: false,  
    home: (Scaffold(  
      body: Container(  
        child: Center(  
          child: Text("Hello world!"),  
        ), // Center  
      ), // Container  
    )), // Scaffold  
  ), // MaterialApp  
);
```

Добавляем BoxDecoration и градиент

Параметр `decoration` у `Container` принимает объект **BoxDecoration** — он описывает визуальное оформление контейнера: цвет, градиент, скругление углов, тень, границу.

Добавьте `decoration` с градиентом внутрь `Container`:



```
body: Container(  
  +decoration: BoxDecoration(),  
  child: Center(  
    child: Text("Hello world!"),  
  ),  
),
```

Далее допишем весь код:

```

debugShowCheckedModeBanner: false,
home: (Scaffold(
  body: Container(
    decoration: BoxDecoration(
      gradient: LinearGradient(
        colors: [
          Colors.white,
          Colors.blue,
          Colors.red,
        ],
      ), // LinearGradient
    ), // BoxDecoration
    child: Center(
      child: Text("Hello world!"),
    ), // Center
  ), // Container
), // Scaffold

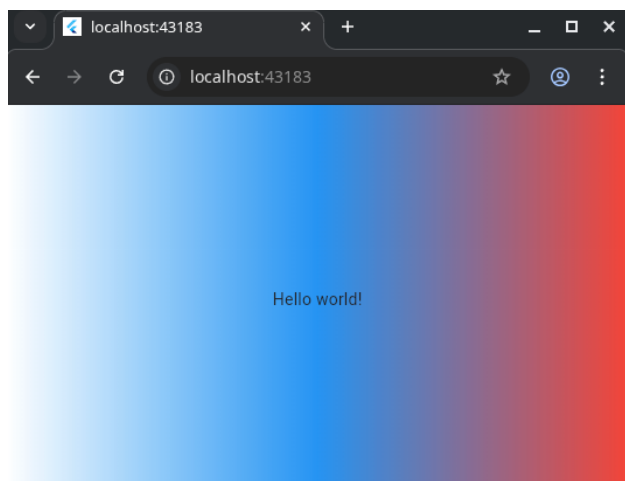
```

Разберём что здесь происходит:

- **BoxDecoration** — объект оформления. Принимает **gradient**, **color**, **borderRadius**, **boxShadow** и другие параметры.
- **LinearGradient** — линейный градиент, то есть плавный переход между цветами по прямой линии (по умолчанию — сверху вниз).
- **colors** — список цветов градиента. Это обобщённый тип **List<Color>** — аналог дженерика **List<Color>** из Kotlin/C#. Flutter плавно интерполирует переход между всеми цветами в списке по порядку.

💡 **LinearGradient** имеет и другие параметры: **begin** и **end** задают направление градиента, **stops** — позиции каждого цвета. Попробуйте добавить **begin: Alignment.topLeft**, **end: Alignment.bottomRight** чтобы сделать диагональный градиент.

Сохраните и перезапустите. Фон стал градиентным.



 **СКРИНШОТ 8:** Скриншот браузера с градиентным фоном. Сохраните в `img/step8_ВашаФамилия.png`

Обратите внимание как растёт дерево виджетов:

MaterialApp

└─ **Scaffold**

└─ **Container** ← оформление фона

└─ **Center**

└─ **Text**

Сделайте коммит:

git add .

git commit -m "Add background color and gradient"

Шаг 11. Направление градиента

Сейчас градиент идёт в случайном направлении — это поведение по умолчанию.

Добавим параметры **begin** и **end** чтобы явно задать направление сверху вниз.

Добавьте два параметра в `LinearGradient` после списка `colors`:

```
colors: [
  Colors.white,
  Colors.blue,
  Colors.red,
],
+ [begin: Alignment.topCenter,
  end: Alignment.bottomCenter,
], // LinearGradient
```

`Alignment` описывает точку на экране по горизонтали и вертикали. Несколько часто используемых значений:

Значение	Позиция
<code>Alignment.topCenter</code>	Верхний центр
<code>Alignment.bottomCenter</code>	Нижний центр
<code>Alignment.topLeft</code>	Верхний левый угол
<code>Alignment.bottomRight</code>	Нижний правый угол
<code>Alignment.center</code>	Центр

Сохраните и выполните **Hot Reload** — градиент теперь идёт строго сверху вниз.

Сделайте коммит:

git add .

git commit -m "Set gradient direction top to bottom"

Шаг 12. Стилизация текста

Сейчас на экране чёрный текст с размером по умолчанию — мелкий и плохо читается на фоне градиента. Применим стили.

Попробуйте самостоятельно

У виджета `Text` есть второй параметр — `style`, который принимает объект `TextStyle()`. Внутри `TextStyle` можно задать размер шрифта и цвет текста.

Попробуйте самостоятельно:

- Размер шрифта: **32**
- Цвет текста: **белый**

Не подглядывайте сразу — поищите нужные параметры через автодополнение VS Code: напишите `TextStyle` и нажмите `Ctrl+Space`, чтобы увидеть список доступных аргументов.

Проверьте себя

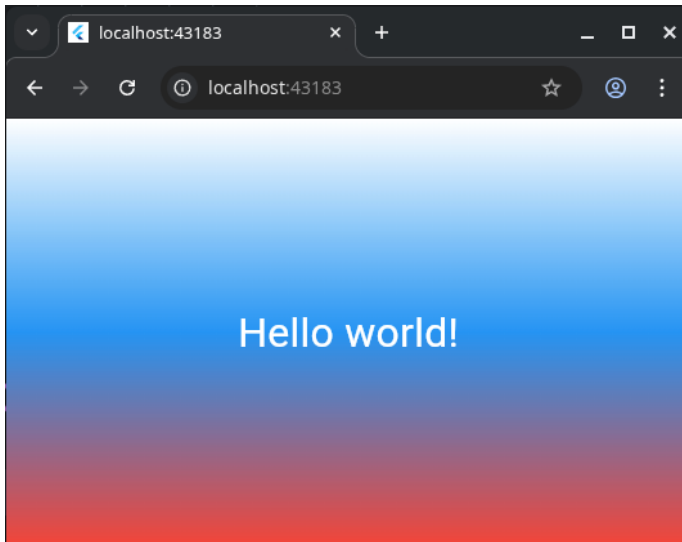
```
child: Center(  
  child: Text(  
    "Hello world!",  
    style: TextStyle(  
      color: Colors.white,  
      fontSize: 32,  
    ), // TextStyle  
  ), // Text  
) // Center
```

`TextStyle` — основной способ оформления текста во Flutter. Помимо размера и цвета он поддерживает множество других параметров:

Параметр	Назначение
<code>fontSize</code>	Размер шрифта
<code>color</code>	Цвет текста
<code>fontWeight</code>	Жирность: <code>FontWeight.bold</code> , <code>FontWeight.w300</code> и т.д.
<code>fontStyle</code>	Начертание: <code>FontStyle.italic</code>
<code>letterSpacing</code>	Межбуквенный интервал
<code>decoration</code>	Подчёркивание, зачёркивание

💡 Обратите внимание: параметр `decoration` у `TextStyle` и параметр `decoration` у `Container` — это разные вещи с одинаковым именем. У `Container` он отвечает за фон и границы, у `TextStyle` — за оформление самого текста (подчёркивание и т.д.).

Сохраните и выполните Hot Reload. **Итог:**



 **СКРИНШОТ 9:** Скриншот браузера с крупным белым текстом на градиентном фоне. Сохраните в `img/step9_ВашаФамилия.png`

Сделайте коммит:

`git add .`

`git commit -m "Style text: size 32, white color"`

Самостоятельные задания

Задание 1. Оформление README.md

Оформите README.md в корне проекта Flutter_Lab2.

Что должно быть в README

1. Заголовок и описание

Название проекта и одно-два предложения о чём он.

2. Информация об авторе

Ваше имя, группа.

3. стек и версии

Укажите чем пользовались: Flutter 3.x.x; Dart 3.x.x; Платформа: Web (Chrome);

IDE: VS Code

4. Скриншот приложения

Вставьте скриншот вашего работающего приложения. Скриншоты уже лежат в папке `img/` — используйте один из них.

5. Как запустить

Опишите шаги, чтобы любой человек мог клонировать репозиторий и запустить проект:

Запуск

1. Клонировать репозиторий
2. Перейти в папку проекта
3. Выполнить `flutter pub get`
4. Запустить командой `flutter run -d chrome`

6. Что изучили

Коротко — 3–5 пунктов что нового узнали в этой лабораторной.

Сделайте коммит:

git add .

git commit -m "added README description"

Задание 2. Приложение «Привет, студент!»

Напишите Flutter-приложение с нуля, которое выводит приветствие с вашим именем.

Требования

Приложение должно отображать на экране:

Привет! Меня зовут [Ваше имя].

Я студент группы [Номер группы].

Технические условия

- Создайте **новый** Flutter-проект: `flutter create hello_student`
- Напишите код **с нуля**, не копируйте из лабораторной
- Используйте только то, что разбирали на паре: `main()`, `runApp()`, `MaterialApp()`, `Text()`
- Проект должен запускаться в Chrome без ошибок

Сделайте коммит:

git add .

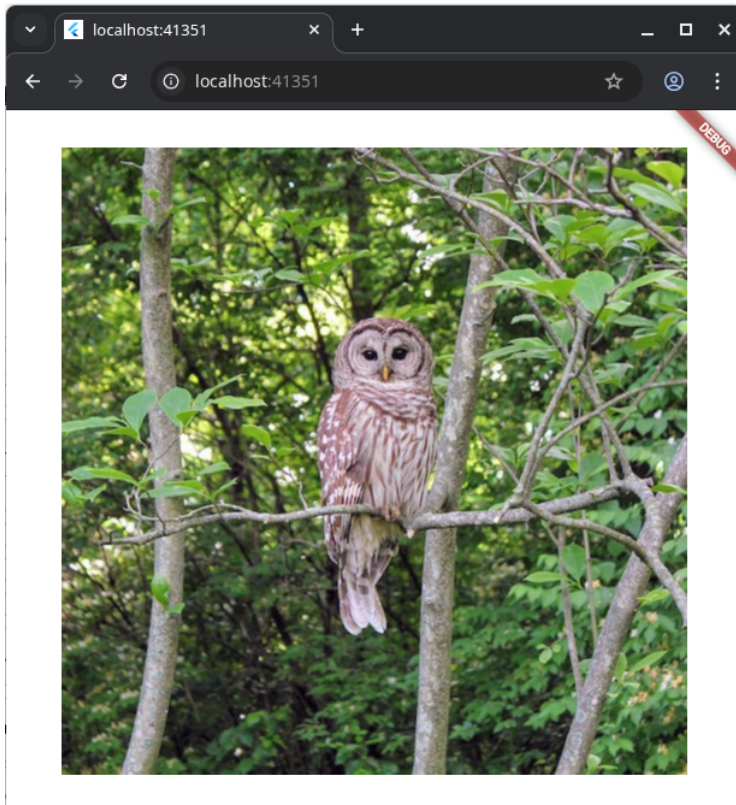
git commit -m "completed task 2"

Задание 3. Image виджет

Попробуйте самостоятельно найти в документации Flutter как вставить картинку в приложение и добавьте любое изображение на экран — например, своё фото или любую картинку из интернета.

<https://docs.flutter.dev/ui/widgets>

Подсказка: ищите виджет, который умеет загружать картинку по ссылке, без скачивания файла. **Пример реализации:**



Сделайте коммит:

git add .

git commit -m "completed task 3 and completed labs"

git push

Полезные ссылки

- [Flutter Documentation](#)
- [Flutter Widget Catalog](#)
- [Flutter tutorial](#)
- [Pub.dev \(пакеты\)](#)